

Round 2 Preparation Guide

Round: Manager Interview 2 — 30 minutes, Tuesday 7:30 PM **Status:** Round 1 cleared **Built from:** the job description, your resume, and what round 1 actually asked

0. How to use this, and the five-day plan

This guide exists because round 1 told us things no amount of research could. You have now seen their style directly, and it is narrower and more predictable than a generic "prepare for a data science interview" scope. Everything here is built from that.

What round 1 asked, in order: 1. Tell me about a model you made → (*HyperVerge*) → layered follow-ups on it 2. Tell me about RAG — why do we need chunking? 3. What's the use of a vector DB over a traditional DB? (*fumbled — wording*) 4. Difference between precision and recall 5. Difference between Random Forest and XGBoost (*fumbled — said "it's supervised"*) 6. A Python question (*solved easily*) 7. A SQL question (*solved easily*) 8. Salesman A sold 40 units, salesman B sold 30 — who's the better salesman?

Five-day plan:

Day	Focus	Sections
Thursday (tonight, 2 hrs)	Your two known fumbles, fixed cold	\$2, then \$3 contrast bank
Friday (3 hrs)	Classical ML mechanics — the layer under the one-liners	\$4, \$7, \$8
Saturday (2 hrs)	Your HyperVerge project, written out three levels deep	\$9
Sunday (3 hrs)	NLP before transformers, then transformers	\$5, \$6
Monday (2 hrs)	Buffer + consolidation. Ambiguity reflex, honest-answer scripts, re-read \$2 and \$3	\$10, \$11, \$12
Tuesday	Drill only (\$12), out loud. Stop by 4 PM.	

Monday is deliberately a buffer. Something always takes longer than planned, and you want slack rather than cramming on interview day.

1. What round 1 revealed about how they interview

Three patterns, and they should shape everything you prepare.

Pattern 1 — they love "difference between X and Y". Three of the five conceptual questions were contrasts. This is the single most reliable thing we know about their style, which is why \$3 of this guide is a contrast bank and why it comes first.

Pattern 2 — they open one project and then dig. You mentioned HyperVerge and they followed up repeatedly on it. This means depth on one project beats breadth across four. Round 2 will almost certainly do this again — either going further on HyperVerge or asking for a different project and drilling that one instead.

Pattern 3 — one deliberately under-specified business question. The salesman question has no correct numerical answer. It is a test of whether you notice that and ask, or whether you confidently pick a number. Expect another one of these, with different nouns.

What they did *not* do: no deep coding, no library syntax, no implementation details, no system design. Your read on this is right, and it means the honest gaps in your hands-on library experience are lower-risk than they'd be elsewhere. Prepare theory and concepts; do not spend Sunday relearning Keras syntax.

What the fumbles tell us. Both were resume-adjacent. XGBoost is on your skills line; vector databases sit under your RAG and FAISS claims. The lesson generalises: **anything written on your resume is a live question**, and a thin answer there costs more than the line gains. \$11 handles the items you cannot deepen in five days.

2. The two fumbles, fixed properly

Do these first tonight. Both have a real chance of resurfacing, and both are cheap to close.

2.1 Random Forest vs XGBoost

You said XGBoost is "supervised". The problem: so is Random Forest, so that sentence doesn't contrast them at all. Here's

the real answer.

The core distinction is bagging versus boosting.

	Random Forest (bagging)	XGBoost (boosting)
How trees are trained	In parallel, independently	Sequentially, one after another
What each tree sees	A bootstrapped sample of rows, random feature subset at each split	The residual errors of the ensemble so far
How predictions combine	Average (regression) or majority vote (classification)	Weighted sum of all trees
What error it reduces	Variance	Bias
Tree depth	Deep trees, individually overfit	Shallow trees, individually weak
Tuning burden	Low — works well out of the box	Higher — learning rate, depth, n_rounds all matter
Noise sensitivity	Robust	More sensitive, because it actively chases errors including noise
Typical accuracy on tabular data	Strong	Usually stronger

The intuition to say out loud:

"Random Forest is bagging — it trains many deep trees in parallel, each on a different bootstrapped sample with random feature subsets, then averages them. Each individual tree overfits, but their errors are uncorrelated enough that averaging cancels them out, so it reduces variance.

XGBoost is boosting — trees are trained sequentially, and each new tree is fitted to the residual errors of the ensemble built so far, using gradient descent on the loss function. So the ensemble progressively corrects its own mistakes, which reduces bias.

Practically, XGBoost usually wins on accuracy for tabular data, but it needs more careful tuning and it's more sensitive to noisy labels — because it's actively chasing errors, and some of those errors are just noise."

Memorise this one line as the fallback: "Bagging — parallel, independent, averaged, reduces variance. Boosting — sequential, each tree fits the previous ensemble's residual, reduces bias."

Likely follow-ups, so be ready: - "What's gradient boosting?" → Each new tree is trained to predict the gradient of the loss function with respect to the current prediction — in the simplest case, the residual. You then add that tree's prediction, scaled by a learning rate, to the ensemble. - "What's the learning rate doing in XGBoost?" → It shrinks each tree's contribution, so no single tree dominates. Lower learning rate needs more trees but generalises better. - "Why does Random Forest use random feature subsets?" → Without it, every tree would pick the same strongest feature at the root and the trees would be highly correlated, which defeats the point of averaging. - "LightGBM or CatBoost — how are they different?" → LightGBM grows leaf-wise instead of level-wise, which is faster but can overfit small datasets. CatBoost handles categorical features natively using ordered target statistics, so it needs less preprocessing.

2.2 Vector DB vs traditional DB

Your instinct was right — storage format and optimisation — but the wording came out circular ("vector DBs store vectors, traditional DBs aren't optimised for vectors"). The missing piece is the indexing mechanism, which is what "optimised" actually means here.

The answer:

"A traditional database indexes for exact match — B-trees, hash indexes. A query like WHERE id = 5 is fast because the index narrows the search space immediately.

A vector database indexes for similarity search — 'find the k nearest vectors to this one.' That's a fundamentally different query pattern, because there's no exact value to look up; you need distance computed against candidates. So it uses approximate-nearest-neighbour structures like HNSW instead of B-trees.

You can store a vector as a column in Postgres — pgvector does exactly that — but without an ANN index, a similarity search means computing distance against every row, which doesn't scale. So the real difference isn't just what's stored, it's that similarity search needs a different kind of index than exact-match lookup does."

Mentioning pgvector is good — it shows you know the boundary isn't absolute. Just always pair it with "needs the ANN index to actually perform", or it sounds like plain Postgres handles this fine.

Likely follow-ups: - "What is HNSW?" → A multi-layer graph. Each vector is a node connected to its nearest neighbours. Top layers are sparse with long-range connections for fast traversal; the bottom layer has every point with dense local

connections for precision. Search starts at the top, greedily hops toward the query, and descends layer by layer. - "Why approximate and not exact?" → Greedy hopping can settle in a local minimum that isn't the true global nearest neighbour. In practice recall is very high, but it isn't mathematically guaranteed the way brute-force flat search is. - "What's an embedding?" → A dense vector representation where semantically similar items land close together in the vector space, produced by a model trained for that property. - "Why not just use keyword search?" → Keyword search can't match paraphrases or synonyms. Embeddings capture meaning. But the reverse is also true — embeddings miss exact identifiers like product codes, which is why hybrid search combining BM25 with dense retrieval exists.

3. The contrast bank — their favourite question format

Three of five conceptual questions in round 1 were "difference between X and Y". Drill these. For each, the goal is a crisp two-sentence answer that names the mechanism, not just the label.

ML contrasts

Supervised vs unsupervised — Supervised learns from labelled data, mapping X to a known y. Unsupervised finds structure in unlabelled data — clusters, lower-dimensional representations, density.

Classification vs regression — Classification predicts a discrete category; regression predicts a continuous number. Different loss functions (cross-entropy vs MSE) and different metrics follow from that.

Overfitting vs underfitting — Overfitting: the model memorises training data including its noise; train error low, validation error high. Underfitting: the model is too simple to capture the real pattern; both errors high.

Bias vs variance — Bias is error from wrong assumptions, so an underfitting model has high bias. Variance is sensitivity to which particular training sample you drew, so an overfitting model has high variance. They trade off against each other as you add model capacity.

Bagging vs boosting — See §2.1. Parallel/independent/averaged/variance versus sequential/residual-fitting/bias.

Training vs validation vs test set — Train is where the model learns parameters. Validation is where you tune hyperparameters and select models. Test is touched once, at the end, to estimate real-world performance — because every look-and-adjust cycle leaks test information into your decisions.

Precision vs recall — Precision: of everything I flagged, how much was actually real ($TP/(TP+FP)$). Recall: of everything actually real, how much did I catch ($TP/(TP+FN)$). They trade off against each other via the decision threshold.

L1 vs L2 regularisation — L1 (Lasso) penalises absolute coefficient values and drives some to exactly zero, so it performs feature selection. L2 (Ridge) penalises squared coefficients, shrinking them smoothly, and handles correlated features better.

Parametric vs non-parametric — Parametric models assume a fixed functional form with a fixed number of parameters (linear regression). Non-parametric models let complexity grow with the data (k-NN, decision trees).

Decision tree vs Random Forest — A single tree is interpretable but unstable; small data changes produce very different trees. Random Forest averages many trees to stabilise that, trading interpretability for accuracy.

k-Means vs DBSCAN — k-Means requires you to specify k, assumes roughly spherical clusters of similar size, and assigns every point to a cluster. DBSCAN is density-based, finds arbitrarily-shaped clusters, doesn't need k, and labels low-density points as outliers.

Batch vs stochastic vs mini-batch gradient descent — Batch computes the gradient over the whole dataset per update: stable but slow. Stochastic uses one sample per update: fast and noisy, and the noise can help escape local minima. Mini-batch uses a small subset, which is the practical compromise and what's used in practice.

Metrics contrasts

Accuracy vs F1 — Accuracy is the fraction correct overall, which is misleading under class imbalance. F1 is the harmonic mean of precision and recall, so it penalises a model that's good at one and bad at the other.

ROC-AUC vs PR-AUC — ROC-AUC plots TPR against FPR. Under imbalance, FPR's denominator contains the huge negative class, so ROC-AUC stays flatteringly high even for a poor model. PR-AUC has no true-negative term anywhere in it, so it can't be fooled that way. Use PR-AUC when the positive class is rare and is the one you care about.

MAE vs RMSE — MAE treats all errors equally and is robust to outliers. RMSE squares errors so large ones dominate. Choose MAE when outliers are noise; RMSE when large errors are genuinely worse.

NLP / LLM contrasts

TF-IDF vs embeddings — TF-IDF is sparse, vocabulary-sized, and based on exact word counts weighted by rarity; it has no notion of meaning, so "car" and "automobile" are unrelated. Embeddings are dense, low-dimensional, and learned, so

semantically similar words land close together.

Sparse vs dense retrieval — Sparse (BM25) matches exact terms in a vocabulary-sized mostly-zero vector. Dense matches meaning in a small fully-populated vector. Sparse wins on exact identifiers and rare terms; dense wins on paraphrases. Hybrid search combines both.

RNN/LSTM vs Transformer — RNNs process sequentially, one token at a time, carrying a hidden state — which can't be parallelised and loses information over long distances. Transformers process all tokens simultaneously with self-attention, so every token can directly attend to every other token regardless of distance, and training parallelises across the sequence.

Encoder vs decoder — Encoders use bidirectional attention (every token sees every other token) and are trained with masked language modelling; used for classification and embeddings, e.g. BERT. Decoders use a causal mask (each token only sees the past) and are trained on next-token prediction; used for generation, e.g. GPT.

Batch norm vs layer norm — Batch norm normalises across the batch dimension, so it depends on batch size and behaves differently at training and inference. Layer norm normalises across features within a single sample, making it batch-size independent — which is why transformers use it, since sequence lengths vary.

RAG vs fine-tuning — RAG injects knowledge at query time by retrieving relevant context, so it handles knowledge that changes and is cheap to update. Fine-tuning changes the model's weights, which is good for teaching *form*, style and task behaviour, but a poor and expensive way to inject facts.

CoT vs self-consistency vs Tree of Thought — CoT is one linear reasoning chain. Self-consistency samples several complete independent chains and majority-votes the final answers. Tree of Thought branches mid-reasoning, evaluates partial paths, prunes weak ones, and can backtrack.

4. Classical ML mechanics — the layer beneath the one-liners

Round 1 asked *how is X different from Y*. Round 2, being a second round, may well ask *how does X actually work*. That's a different depth, and this is where your guides were thinnest. Friday's material.

4.1 How a decision tree actually splits

At each node, the tree tries every feature and every possible threshold, and picks the split that makes the resulting groups as **pure** as possible — meaning each child node contains mostly one class.

Purity is measured two common ways:

Gini impurity = $1 - \sum(p_i^2)$, where p_i is the proportion of class i in the node. - All one class $\rightarrow$ Gini = 0 (perfectly pure) - 50/50 two-class split $\rightarrow$ Gini = $1 - (0.25 + 0.25) = 0.5$ (maximum impurity)

Entropy = $-\sum p_i \cdot \log_2(p_i)$ - All one class $\rightarrow$ entropy = 0 - 50/50 split $\rightarrow$ entropy = 1

Information gain = entropy(parent) - weighted average entropy(children). The tree picks the split with the highest information gain.

Gini and entropy almost always choose similar splits; Gini is slightly cheaper to compute, which is why it's the common default.

Why trees overfit: left unconstrained, a tree keeps splitting until every leaf is pure — which means it has memorised the training data. That's what `max_depth`, `min_samples_leaf` and `min_samples_split` exist to prevent.

For regression trees: instead of Gini/entropy, splits minimise variance (or MSE) within each child node, and the prediction at a leaf is the mean of the training targets that landed there.

4.2 Gradient descent

The algorithm that trains most models. You want the parameters that minimise the loss function.

1. Start with random parameters
2. Compute the **gradient** of the loss with respect to each parameter — the direction of steepest increase
3. Step in the **opposite** direction (downhill), scaled by the **learning rate**
4. Repeat until the loss stops improving

$\theta = \theta - \alpha \cdot \nabla L(\theta)$, where α is the learning rate.

Learning rate is the critical hyperparameter. Too high and you overshoot the minimum and diverge or oscillate. Too low and training takes forever or stalls in a plateau.

Variants: batch (whole dataset per update — stable, slow), stochastic (one sample — fast, noisy), mini-batch (small subset

— the practical standard). Momentum adds a fraction of the previous update to the current one, helping push through flat regions and dampening oscillation. Adam combines momentum with per-parameter adaptive learning rates and is the common default.

4.3 Logistic regression

Despite the name, it's classification. It computes a weighted sum of features (like linear regression), then passes it through the **sigmoid** function to squash it into (0, 1) as a probability:

$$\sigma(\mathbf{z}) = 1 / (1 + e^{-z}), \text{ where } z = \mathbf{w} \cdot \mathbf{x} + b$$

Trained by minimising **binary cross-entropy** loss (also called log loss) via gradient descent. The decision threshold — usually 0.5, but tunable — converts that probability into a class.

Why it's still used: coefficients are directly interpretable (a positive weight means that feature pushes toward the positive class), it's fast, and it's the right baseline before reaching for anything complex.

4.4 k-Nearest Neighbours

No training phase at all — it stores the training data. To predict, find the k closest training points to the new point (usually by Euclidean distance) and take the majority class (or the mean, for regression).

Key points: k is a hyperparameter — small k is noisy and overfits, large k oversmooths. Features must be scaled, or whichever feature has the largest numeric range dominates the distance calculation. It suffers badly from the **curse of dimensionality**: in high dimensions, all points become roughly equidistant and "nearest" stops being meaningful.

4.5 Support Vector Machines

Finds the hyperplane that separates classes with the **maximum margin** — the largest possible distance to the nearest point of each class. Those nearest points are the **support vectors**, and they're the only points that actually determine the boundary.

The kernel trick: when data isn't linearly separable, a kernel function (RBF, polynomial) implicitly maps it into a higher-dimensional space where it is separable — without ever computing the coordinates in that space explicitly.

4.6 Naive Bayes

Applies Bayes' theorem to classification: $P(\text{class} | \text{features}) \propto P(\text{features} | \text{class}) \cdot P(\text{class})$.

The "naive" part is assuming all features are **conditionally independent** given the class. That assumption is almost always false in reality, but the model works surprisingly well anyway — especially on text, where it's extremely fast and a strong baseline.

5. NLP before transformers

You said "NLP transformers" in your plan. This is the layer underneath, and it's a reasonable thing for them to probe — particularly "why did transformers replace RNNs", which is a contrast question and therefore in their favourite format.

5.1 Text representation, historically

Bag of Words — represent a document as counts of each vocabulary word. Loses all word order ("dog bites man" = "man bites dog"). Sparse and vocabulary-sized.

TF-IDF — weight each word by term frequency $\times$ inverse document frequency, so common words like "the" are down-weighted and rare distinctive words are up-weighted. Still sparse, still order-blind, still no notion of meaning.

Word2Vec / GloVe — the shift to **dense learned embeddings**. Each word becomes a fixed vector (say 300 dimensions) where semantically similar words land close together. Trained on the principle that words appearing in similar contexts have similar meanings.

Word2Vec has two training setups: **CBOW** (predict the target word from its surrounding context) and **Skip-gram** (predict the surrounding context from the target word). Skip-gram works better for rare words.

The famous property: vector arithmetic captures relationships — king – man + woman $\approx$ queen.

The limitation that mattered: these are **static** embeddings. "Bank" gets exactly one vector whether it means a riverbank or a financial institution. Context can't change the representation.

5.2 Tokenization

Models don't see words, they see **tokens**. Modern models use **subword** tokenization (BPE, WordPiece, SentencePiece) which splits rare words into pieces while keeping common words whole.

Why subword: word-level tokenization can't handle words it never saw in training (out-of-vocabulary). Character-level handles anything but makes sequences very long. Subword is the compromise — a vocabulary of ~30k-100k pieces covers any input, including typos and new words, by decomposing them.

This is why token counts don't match word counts, and why unusual words cost more tokens.

5.3 RNNs and LSTMs

RNN — processes a sequence one token at a time, maintaining a **hidden state** that carries information forward. The same weights are applied at each step.

The problems: - **Sequential by nature** — you can't compute step 5 before step 4, so training can't parallelise across the sequence. Slow. - **Vanishing gradients over long sequences** — gradients shrink multiplicatively as they propagate back through many time steps, so the model effectively can't learn long-range dependencies.

LSTM — adds **gates** (input, forget, output) and a separate **cell state** that information can flow along relatively unchanged. The gates learn what to keep, what to discard, and what to output. This substantially mitigates vanishing gradients and extends usable context length.

GRU — a simplified LSTM with two gates instead of three. Fewer parameters, often comparable performance.

5.4 Why transformers replaced them

This is the contrast most likely to be asked, so have it clean:

	RNN / LSTM	Transformer
Processing	Sequential, one token at a time	All tokens simultaneously
Parallelisation	Impossible across the sequence	Full parallelisation during training
Long-range dependencies	Degrade with distance; information must pass through every intermediate step	Direct — any token attends to any other token in one step
Path length between two tokens	O(n)	O(1)
Cost	Linear in sequence length	Quadratic in sequence length

"RNNs process sequentially and carry information through a hidden state, which means two things: training can't parallelise across the sequence, and information between distant tokens has to survive passing through every intermediate step, so long-range dependencies degrade. Transformers use self-attention, where every token directly attends to every other token in a single operation — so the path between any two positions is constant length regardless of distance, and the whole sequence processes in parallel. The cost is that attention is quadratic in sequence length, which is why long context is expensive."

6. Transformers — condensed

You haven't studied this, and it's the architecture underneath everything you do daily. Sunday's priority.

6.1 Self-attention

Each token is projected into three vectors: **Query** (what am I looking for), **Key** (what do I contain), **Value** (what do I contribute).

Attention weights come from the dot product of one token's Query with every token's Key, scaled, softmaxed, and then used to take a weighted sum of the Values:

Attention(Q, K, V) = softmax(QK^s / √d_k) V

Why divide by √d_k? Dot products of high-dimensional vectors have variance proportional to d_k. Without scaling, the values entering the softmax are large, the softmax saturates into a near-one-hot distribution, and gradients vanish. Dividing by √d_k keeps the variance roughly constant regardless of dimension.

This is a commonly asked question and the answer is one sentence. Know it.

6.2 Multi-head attention

Run h attention operations in parallel on lower-dimensional projections, then concatenate and project back.

Why: a single attention operation produces one weighted average, which blends all relationships together. Multiple heads let different heads specialise — one might track syntactic dependencies, another coreference, another positional patterns — rather than averaging everything into one blurry pattern.

6.3 Positional encoding

Self-attention is **permutation-invariant** — it has no inherent notion of order, because it's computing pairwise relationships, not reading left to right. So position has to be injected explicitly.

- **Sinusoidal** — fixed trigonometric functions, from the original paper
- **Learned** — position embeddings trained like any other parameter
- **RoPE (rotary)** — encodes relative position by rotating Q and K vectors; extrapolates better to longer sequences
- **ALiBi** — adds a distance-based linear bias directly to attention scores

6.4 The full block

Multi-head attention → residual connection → layer norm → feed-forward network (typically 4× wider, GELU activation) → residual connection → layer norm.

Residual connections let gradients flow directly past each sublayer, which is what makes very deep stacks trainable.

Pre-norm vs post-norm: modern models normalise *before* each sublayer rather than after, because it trains more stably at depth.

6.5 Architecture families

Type	Attention	Trained on	Used for	Example
Encoder-only	Bidirectional	Masked language modelling	Classification, embeddings	BERT
Decoder-only	Causal mask	Next-token prediction	Generation	GPT, Llama, Claude
Encoder-decoder	Both + cross-attention	Seq2seq	Translation	T5

Causal mask — in a decoder, each position is prevented from attending to future positions, so the model can't cheat by seeing the answer it's supposed to predict.

6.6 Inference cost

Quadratic complexity — attention is $O(n^2 \cdot d)$ in sequence length, because every token attends to every other token. This is why long context is expensive.

KV cache — during generation, the Keys and Values for all previous tokens would be recomputed at every new token. Caching them makes per-token cost linear instead of quadratic. The trade-off: cache memory grows with sequence length × layers × heads, which is why **inference is typically memory-bound rather than compute-bound**.

MQA / GQA — Multi-Query and Grouped-Query Attention share Key/Value projections across attention heads specifically to shrink that cache.

6.7 Decoding parameters

Temperature scales the logits before the softmax — 0 is greedy and deterministic, higher is more random. **Top-k** samples from the k most likely tokens. **Top-p (nucleus)** samples from the smallest set whose cumulative probability exceeds p.

Use temperature 0 for extraction and classification; reserve higher values for genuinely creative generation.

7. Clustering and unsupervised learning

Thin in your existing material, and a plausible "how does it work" target.

7.1 k-Means

1. Choose k, initialise k centroids (randomly, or with k-means++ for better starting points)
2. **Assign** each point to the nearest centroid
3. **Update** each centroid to the mean of the points assigned to it
4. Repeat 2-3 until assignments stop changing

Minimises: within-cluster sum of squared distances (inertia).

Limitations to name: you must choose k in advance; it assumes roughly spherical, similarly-sized clusters; it's sensitive to initialisation (hence k-means++ and multiple restarts); and it assigns every point to a cluster, so outliers get forced somewhere.

Choosing k: the **elbow method** (plot inertia against k, look for the bend where additional clusters stop helping much) or **silhouette score** (measures how similar each point is to its own cluster versus the nearest other cluster, ranging -1 to 1).

7.2 DBSCAN

Density-based. Two parameters: **eps** (neighbourhood radius) and **min_samples** (how many points must be within eps to count as dense).

Points are classified as **core** (at least min_samples neighbours within eps), **border** (within eps of a core point but not itself core), or **noise** (neither). Clusters grow by chaining core points together.

Advantages: finds arbitrarily-shaped clusters, doesn't need k specified, explicitly identifies outliers. **Disadvantages:** struggles when clusters have very different densities, and eps is hard to choose in high dimensions.

7.3 PCA

Finds the directions (**principal components**) of maximum variance in the data, which are orthogonal to each other, and projects the data onto the first few.

Why: dimensionality reduction, decorrelating features, visualisation, and speeding up downstream models.

Key points: components are ordered by how much variance they explain, so you pick enough to retain (say) 95% of total variance. Data must be standardised first, or high-variance-scale features dominate. The resulting components are linear combinations of original features, which means they're generally **not interpretable** — a real cost.

8. Statistics essentials

Lightweight, but cheap insurance.

Mean vs median — mean is sensitive to outliers; median is robust. For skewed data (salary, latency, session length), median is the honest summary.

Variance and standard deviation — variance is the average squared deviation from the mean; standard deviation is its square root, back in the original units.

Normal distribution — symmetric and bell-shaped. The **68-95-99.7 rule**: that proportion of data falls within 1, 2 and 3 standard deviations of the mean.

Skew — right/positive skew has a long right tail and mean > median. Left skew is the reverse.

Central Limit Theorem — the distribution of **sample means** approaches normal as sample size grows, regardless of the underlying distribution's shape. This is why you can construct confidence intervals around a mean even for skewed data. Standard error = $\sigma/\sqrt{n}$, so quadrupling your sample only halves your uncertainty.

p-value — the probability of observing data at least this extreme **if the null hypothesis were true**. It is *not* the probability that the null hypothesis is true. That misstatement is the single most common statistics error in interviews.

Type I vs Type II error — Type I is a false positive (claiming an effect that isn't there), rate α . Type II is a false negative (missing a real effect), rate β . Power = $1 - \beta$.

Confidence interval — a 95% CI means that if you repeated the experiment many times, 95% of the intervals constructed this way would contain the true parameter.

Correlation vs causation — correlation measures linear association only. Confounders, reverse causality, and selection bias all produce correlation without causation.

Why normalise/standardise — so features with large numeric ranges don't dominate distance calculations or gradient updates. Needed for k-NN, SVM, logistic regression and neural networks; not needed for tree-based models, since splits are scale-invariant.

Simpson's paradox — a trend present in every subgroup reverses when the groups are combined. Worth knowing by name, because it's the formal version of "always segment before concluding" — which connects directly to §10.

9. Your HyperVerge project — three levels deep

Round 1 opened with "tell me about a model you made", you gave HyperVerge, and they followed up. **Round 2 will very likely go further on this, or ask for a different project and drill that one instead.** This section is Saturday's work, and it's the highest-value two hours in the whole plan.

9.1 Why this matters more than any concept

A past candidate for this exact role reported that the key thing asked was to explain a project in detail — not just the technical side, but the thought process, the challenges, and how they were solved. Your round 1 matched that exactly.

Concepts get you past a question; project depth gets you past a round.

9.2 Write out answers to all of these, in full sentences

Don't just think them. Writing is what makes them retrievable under pressure.

The problem - What was the business or research problem, in one sentence a non-technical person would understand? - Who needed the output, and what decision did it feed? - Why was it worth solving — what was the cost of not solving it?

The data - What was the data? How many rows/images/samples? - Where did it come from? Was it labelled already, or did labelling have to happen? - **Was it balanced?** How many of each class? (*Critical — see 9.3*) - What was wrong with it — missing values, noise, mislabels, duplicates?

The approach - What did you try first, and why that? - What did you try that **didn't** work, and how did you know? - Why TensorFlow/Keras rather than scikit-learn — was it a neural network, and if so why was a neural network the right call? - What were the features, or if it was images, what preprocessing?

The evaluation - What exactly does the **97%** measure? Accuracy on a held-out test set? - How big was that test set? - What was the **baseline** you improved on? ("over baseline" is on your resume — be ready to say what the baseline was and how it was computed.) - Did you look at precision and recall, or only accuracy?

The hard parts - What was genuinely difficult? Name one specific thing. - What broke, and how did you debug it? - What was the moment you were stuck, and what unstuck you?

The retrospective - What would you do differently now? - What did you learn that changed how you work?

9.3 The one answer that matters most

"Was the dataset balanced?" — This is the question most likely to catch you, because your resume quotes **accuracy** and you now know accuracy is the wrong metric under imbalance.

If it was balanced: say so plainly, and accuracy was a reasonable choice. Easy.

If it was imbalanced, or you're not sure: the strongest possible answer is honest reflection, not defence:

"It was imbalanced — roughly [X:Y]. At the time I reported accuracy because that's what we were tracking, but preparing for this conversation I've been thinking about it, and accuracy wasn't really the right metric there. PR-AUC or precision and recall at the operating threshold would have told a truer story, because with that imbalance a model could score well on accuracy while being poor on the minority class. If I did it again that's what I'd report."

That answer is **better** than having gotten it right originally. It demonstrates metric judgement, intellectual honesty, and growth — three things a manager round is explicitly testing. Do not try to defend accuracy as the right choice if it wasn't.

9.4 Decide now: HyperVerge or the ETL conversion agent?

You have a second, arguably stronger project — the LLM-agent system that converts ETL pipelines end-to-end, which you designed and coded, shipped 4-5 times, with an observability package tracking LLM cost on GCP.

Case for leading with HyperVerge again: they already know it, continuity looks consistent, and it's a clean classical-ML story that answers "tell me about a model you built" literally.

Case for leading with the conversion agent: it's more impressive, more recent, more aligned with AI Engineer, and it's work you can discuss without any rust. It also shows range beyond what they already probed.

My suggestion: have both ready. If they ask "tell me about another project", go to the conversion agent. If they go deeper on HyperVerge, you're prepared. But if you lead with the conversion agent, be ready for these specifically: - How does the agent actually parse and map the source pipeline's logic? - **How do you verify the conversion is correct?** (*Most important — "AI writes code" immediately raises "how do you trust it"*) - What happens when it gets it wrong — retries, human review? - Why an LLM agent rather than deterministic rule-based translation? - What's in the observability package — what metrics, how is cost computed?

If you don't yet have a strong answer to the verification question, that's worth thinking through before Tuesday, because it's the obvious follow-up and the one that most tests whether you've engineered this or just wired it together.

10. The ambiguous business question — expect another one

The salesman question (A sold 40, B sold 30, who's better?) has **no correct numerical answer**. That's the point. They're testing whether you notice the question is under-specified, or whether you confidently pick the bigger number.

10.1 The reflex to build

Name what's undefined, before answering. Then reason toward a framework, then commit to a view with stated assumptions.

Four moves, in order: 1. **"Better how?"** — define the metric. Units sold? Revenue? Margin? Conversion rate? Retention of the customers they sold to? 2. **"What's held constant?"** — same time period? Same territory? Same product mix? Same number of leads worked? 3. **"What's the denominator?"** — 40 units from 400 leads is worse than 30 from 100. Raw output without opportunity is meaningless. 4. **Commit** — "On the information given, A sold more. But I wouldn't call A the better salesman without knowing conversion rate and territory, because those usually explain more of the variance than individual skill does."

10.2 The salesman question, answered well

"It depends what we mean by better, and I'd want a few things before answering. Over the same period? Same territory and market size — because a salesperson in a dense urban patch has a very different opportunity set than one covering a rural region. Same product, or do their products have different margins and different difficulty? And most importantly, how many leads did each work — 40 units from 400 leads is a 10% conversion rate, 30 from 100 is 30%, which would make B substantially better at selling even though A sold more.

On the raw numbers given, A sold more units. But units sold is an output measure, and what I'd actually want to evaluate salesmanship is conversion rate normalised for territory difficulty and product mix."

10.3 Other likely variants

Practise the same reflex on these: - "Campaign A has a higher click-through rate than campaign B — which is better?" (*CTR vs conversion vs cost per acquisition; who was each shown to?*) - "Course X has higher completion than course Y — which is the better course?" (*Who enrolls in each? Course length? Difficulty? Prior-knowledge requirement?*) - "Our conversion rate dropped 10% this month — what happened?" (*First rule out instrumentation and tracking changes, then segment by channel, geography, device, cohort before theorising.*) - "Model A has 95% accuracy, model B has 92% — which do you deploy?" (*Class balance? Precision/recall profile? Latency? Cost? What does an error cost in each direction?*)

The last one is particularly plausible given they already asked about metrics, and it lets you deploy everything from \$3.

11. Honest-answer scripts for thin resume claims

Round 1 proved the pattern: a resume line invites a question, and a thin answer there costs more than the line gains. You cannot rebuild five years of library experience in five days, and **attempting to fake it is how the XGBoost answer happened**. A calm, honest, specific answer costs you very little. A confident wrong answer costs you a lot.

Have these ready so you don't improvise under pressure.

TensorFlow / Keras

"I used them for the HyperVerge project — that was a while back now, so I'd be honest that I'm rusty on the specifics of the API. Conceptually I'm clear on what they do: Keras as the high-level layer for defining and training networks, TensorFlow underneath it. If I picked it up again I'd be productive quickly, but I wouldn't claim day-to-day fluency right now."

scikit-learn

"I've used it for classical ML work, mostly during college and internships. I know the structure — estimators with fit/predict, pipelines, the model selection utilities — but my recent work has been LLM systems, so that's where my current depth is."

EDA / Statistical Modeling

"EDA for me is the step before modelling — understanding distributions, missingness, outliers, correlations, and whether the data actually supports the question being asked. I've done that work, though more pragmatically than formally."

Docker / CI-CD

"I've worked with CI/CD pipelines operationally — written and modified YAML configs, deployed through them. I'd be honest that I've leaned on existing templates and patterns rather than designing pipeline architecture from scratch."

GCP

"My GCP usage has been hands-on but operational — deploying and running services, hosting packages, managing the environments our systems run in. I haven't done infrastructure design or cost architecture, so that's a gap I'd be learning on the job."

Matplotlib

"I use it for basic visualisation when exploring data — I'm not doing anything sophisticated with it."

The general principle: say what's true, say it briefly and without apology, name the boundary clearly, and move on. Don't over-explain or sound like you're confessing. "That's a gap I'd be learning on the job" is a completely normal thing for someone fourteen months into their career to say, and it builds trust rather than costing it.

And crucially — redirect to strength. After an honest limit, it's fine to add one line: *"Where I'd say I'm genuinely strong is production LLM systems — RAG, agent orchestration, the evaluation and observability around them."* That's true, it's relevant to the role, and it moves the conversation to ground you own.

12. The drill

Answer each out loud before reading on. Do this Monday and again Tuesday. **Retrieval practice is what makes things stick under pressure** — reading the guide a third time is not.

The two fumbles

1. **Random Forest vs XGBoost?** → Bagging vs boosting. Parallel independent trees averaged, reduces variance / sequential trees fitting residuals, reduces bias.
2. **What's gradient boosting?** → Each new tree fits the gradient of the loss w.r.t. current predictions — the residual — added to the ensemble scaled by a learning rate.
3. **Vector DB vs traditional DB?** → Exact-match indexing (B-tree, hash) vs similarity indexing (ANN, HNSW). Postgres + pgvector can store vectors but needs the ANN index to search at scale.
4. **What's HNSW?** → Multi-layer graph, sparse long-range at top for fast traversal, dense local at bottom for precision, greedy descent toward the query.

Metrics

5. **Precision vs recall?** → Of what I flagged, how much was real / of what's real, how much I caught.
6. **When is accuracy a bad metric?** → Under class imbalance. 1% positives, predict all-negative, 99% accuracy, useless model.
7. **What would you use instead?** → Precision/recall/F-beta by cost structure, PR-AUC over ROC-AUC, confusion matrix at the operating threshold, precision@k if the output is a ranked list.
8. **Why PR-AUC over ROC-AUC under imbalance?** → ROC-AUC's FPR denominator contains the huge negative class, so it stays optimistically high. PR-AUC has no TN term.
9. **What's F1?** → Harmonic mean of precision and recall; penalises being good at one and bad at the other.

ML concepts

10. **What's overfitting, how do you detect it, how do you fix it?** → Memorising noise; train error low and validation high; regularise, more data, less capacity, early stopping — and check for leakage.
11. **Bias vs variance?** → Wrong assumptions vs sensitivity to the sample. Underfit vs overfit.
12. **Why split into train/validation/test?** → Parameters, hyperparameters, and an untouched final estimate. Every look-and-adjust leaks test info into decisions.
13. **What's cross-validation and why?** → k folds, train k times, average. More reliable than one split and uses all the data.
14. **How does a decision tree decide where to split?** → Tries features and thresholds, picks the split maximising purity — Gini or information gain from entropy.

15. **What's gradient descent?** → Compute the gradient of loss w.r.t. parameters, step downhill scaled by the learning rate, repeat.
16. **L1 vs L2?** → L1 zeroes coefficients and selects features; L2 shrinks smoothly and handles correlated features.
17. **What's data leakage?** → Information unavailable at prediction time entering training. Scaling before splitting, post-outcome features, temporal leakage.
18. **How do you handle imbalanced data?** → Class weights first, threshold tuning, then resampling — and resample inside CV folds only.
19. **How does k-means work?** → Initialise k centroids, assign to nearest, update to the mean, repeat until stable. Choose k by elbow or silhouette.

NLP and transformers

20. **Why did transformers replace RNNs?** → RNNs are sequential (no parallelisation) and long-range info degrades through intermediate steps. Attention gives constant path length between any two tokens and full parallelisation.
21. **What's self-attention?** → $\text{softmax}(QK^T/\sqrt{d_k})V$ — each token's query against all keys, softmaxed into weights over values.
22. **Why divide by $\sqrt{d_k}$?** → Dot-product variance scales with d_k ; without scaling the softmax saturates and gradients vanish.
23. **Why multi-head?** → Different heads specialise on different relationships rather than averaging everything into one pattern.
24. **Why layer norm not batch norm?** → Batch-size independent and handles variable sequence lengths.
25. **Why is positional encoding needed?** → Attention is permutation-invariant; it computes pairwise relationships with no inherent notion of order.
26. **Encoder vs decoder?** → Bidirectional + masked LM for understanding/embeddings; causal mask + next-token prediction for generation.
27. **What's the KV cache and why does it matter?** → Caches previous keys/values so generation is linear not quadratic; dominates inference memory, which is why inference is memory-bound.
28. **TF-IDF vs embeddings?** → Sparse vocabulary-sized exact word counts weighted by rarity, no meaning / dense learned low-dimensional vectors where similar meanings land close.

RAG

29. **Why do we need chunking?** → Documents exceed context and embedding limits; retrieval needs granular units. Too small loses context, too large blurs the embedding and kills precision.
30. **RAG vs fine-tuning?** → RAG for knowledge that changes, injected at query time. Fine-tuning for form and behaviour, not facts.
31. **What's hybrid search?** → BM25 sparse plus dense retrieval, fused with RRF. Sparse catches exact identifiers that embeddings miss; dense catches paraphrases that keywords miss.

Business judgement

32. **Salesman A sold 40, B sold 30 — who's better?** → §10.2. Name what's undefined, then commit with assumptions stated.
33. **Model A is 95% accurate, B is 92% — which ships?** → Class balance, precision/recall profile, latency, cost of each error direction.

Project

34. **Tell me about a model you built.** → HyperVerge, three levels deep per §9.
35. **Was the dataset balanced?** → §9.3. Honest, and if imbalanced, the reflective answer beats the defensive one.
36. **What was the baseline?** → Know this number.
37. **What would you do differently?** → Have a real answer. Metric choice is a strong one.

Closing note

You cleared round 1 without having studied ML or transformers at all. That tells you the bar is passable and your project experience carries real weight. Five days is enough to close the two gaps that actually showed, add the depth layer beneath the one-liners, and walk in without a soft spot.

Two things to do that aren't reading: 1. **Write the HyperVerge answers out in sentences** on Saturday. Writing is what makes them available under pressure. 2. **Do the drill out loud** Monday and Tuesday — not re-read, answered aloud.

And pin down your 97% and your baseline. A precise number on a resume invites a precise question, and that's the one line most likely to be probed.

Good luck Tuesday.